TECHNICAL ASSIGNMENT

Intake Triage Agent

Design, evaluation and production plan for automated enquiry triage

Harsha Dindigal · October 2026 · github.com/harshadindigal-dev/AIVCInterview · All companies and people are fictional

1 Summary

The firm receives 40 to 60 enquiries a week through its website. An analyst reads each one, decides the service line and complexity, and passes it to a team lead. That takes about eight hours a week, and mistakes slip through.

The prototype automates this triage. A language model reads each enquiry and judges the service line, complexity and urgency. The CRM adds what the firm already knows about the client. A short set of rules, owned by the firm rather than the model, decides who receives it. Anything uncertain, conflicted or suspicious goes to a person, with the reason stated. Every decision can be traced, and every correction is recorded.

Testing. Three models were run on 50 hand-written enquiries. Half were deliberately difficult: vague requests, conflicts of interest, manipulation attempts, and work spanning several practices. Every enquiry that needed a person reached one, both manipulation attempts were caught, and every automatic route was correct.

Model	Needed a person, reached one	Automatic routes correct	Service line	Complexity	Cost per 1,000
Claude Sonnet 5.5 (recommended)	8 of 8	37 of 37	100%	90%	$40
Claude Haiku 5.5	8 of 8	37 of 37	100%	66%	$2.50
Gemini 3.8 Flash	8 of 8	37 of 37	100%	73%	$19

These results flatter the system. The test enquiries were written alongside the firm's definitions, so they show the model following rules more than coping with real-world variety. The clearest weakness is complexity, which every model overstates.

Recommendation. Run the system in shadow mode alongside the analyst, then as suggestions that team leads confirm with a click, and only then route clear-cut enquiries automatically. On the test set, about one enquiry in six needed a person. If that holds, the analyst's eight hours a week fall to one or two.

2 How it works

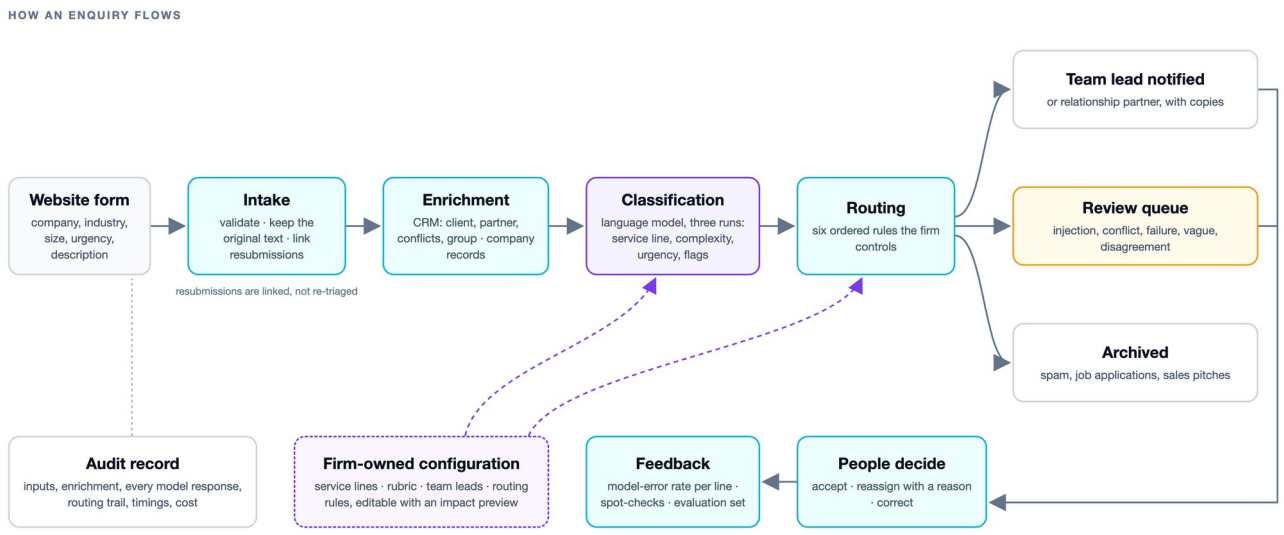


Figure 1. How an enquiry moves through the system

Intake. The submission is validated and the original wording kept exactly. A near-identical resubmission within a week is linked to the first, not triaged twice.

Enrichment. The CRM shows whether the company is a client, its relationship partner, the services already provided, the last engagement and any conflict of interest. Subsidiaries are resolved to their group, so a newly acquired business

inherits its parent's partner and conflicts. Company records add size, revenue and ownership, and mismatches with what the client stated are flagged.

Classification. The model's instructions are generated from two documents the firm owns: the service-line definitions and the complexity and urgency rubric. The enquiry is treated as untrusted text, and any instructions inside it are flagged. The model returns a structured result: service lines, complexity with its rationale, urgency, key facts, questions for the client, risk flags and a one-line summary. Each enquiry is classified three times, and disagreement sends it to review; models are poor judges of their own confidence, so the system compares answers instead. A failed response is retried once, then passed to a person.

Routing is kept away from the model. These rules run in order, and the first that applies decides:

1. Possible manipulation, a conflict of interest or a failed classification goes to a person.
2. Spam, job applications and sales pitches are archived.
3. Existing clients go to their relationship partner, with the service-line lead copied.
4. Vague or out-of-scope enquiries, and those where the runs disagreed, go to the review queue.
5. Complex, urgent work goes to the service-line lead, with the practice head copied.
6. Everything else goes to the service-line lead.

The model can only choose from the firm's service lines, so it cannot name a person or invent a destination. Every rule checked is recorded and shown with the enquiry, along with all three model responses, timings and cost.

Keeping the logic in the firm's hands. Definitions, rubric, team leads and rules live in configuration. Practice operations can change them on a configuration screen. Before a change is saved, it shows which past enquiries would have been routed differently. Changes are logged, reversible, and recorded against every decision they produce.

Model and API choices

Models are accessed through OpenRouter. It gives one interface across vendors, automatic fallback to a backup model, and the actual cost of each call. Claude Sonnet 5.5 is the default because it judged complexity best and explained itself most clearly; at ten enquiries a day the cost difference is small. Claude Haiku matched it on routing and safety at a sixteenth of the cost, which matters if volume grows.

The trade-off is a third party in the path of client data. In production I would restrict it to zero-retention providers, cover it in the data processing agreement and strip personal details before sending. Once a model is chosen, I'd call that provider directly from the firm's own cloud account, which is a small change because the model sits behind one interface.

Integration points

Component	Prototype	Production
Client and company data	Sample CRM of 28 companies, with group structures	CRM (Dynamics or Salesforce) and Companies House or D&B
Conflict check	A field in the sample CRM	The firm's conflicts system, checked before routing
Language model	OpenRouter	Zero-retention providers only, or the chosen provider directly
Notifications	Rendered and logged	Email and Slack, with signed accept and reassign links
Storage and access	SQLite; open access	Postgres; single sign-on, with configuration limited to administrators

Next step: on-demand external enrichment. Most enquiries come from prospects the CRM doesn't know. For those, and for complex or high-value work, the system could look outward:

- **Exa,** for recent news and filings about a company.
- **Parallel,** for cited answers to questions such as "who owns this company?".
- **Clay,** for headcount and revenue from many data providers.

Only the company name and web domain would be sent, never the enquiry or personal details. Results would be labelled as external, and they could send an enquiry for review but never route it on their own.

3 What breaks first, how we'd know, and the fallback

What breaks first: a confident wrong answer. When an enquiry doesn't fit the firm's categories neatly, the model files it under the nearest service line, all three runs agree, and nothing sends it for review. On the test set the runs never disagreed once. New kinds of work, such as a new practice area, are the sharpest risk because they get forced into an existing line without warning. The second weakness is complexity, which is overstated, mostly because any mention of a board is read as board-level work. This rarely changes routing, but it misleads team leads about effort.

Not every mistake costs the same. A missed conflict of interest, or acting on instructions hidden in an enquiry, must never happen quietly, so both are blocked by fixed rules whatever the model says. A wrong service line is costly but recoverable, and monitoring is built to catch it quickly. A complexity rating one level out costs a person a minute. The system leans towards sending more to review rather than less.

What I'd monitor

Signal	Alert when	Owner
Model errors per service line (reassigned because the line was wrong, or corrected; workload moves are recorded separately)	Above 10% for any line over two weeks	Practice operations, weekly
Accuracy of automatic routes, from a 10% weekly spot-check	Below 95%	Analyst, weekly
Share sent to review, and the reasons	Outside 10% to 30%	Practice operations, weekly
Age of items in the review queue	Older than one working day	Analyst, daily
Out-of-scope and vague enquiries; service-line mix	Double, or half, the usual level	Practice operations, monthly
Failures, response time, cost; test-suite results	Failures above 2% a day, or any safety case missed	Engineering, daily and on every change

When the model gets it wrong

1. **Before anything is sent.** Suspected manipulation, conflicts and failed classifications always go to a person. The model can't send anything to an unexpected destination. If the model is unavailable, the system retries, then tries a backup model, then the review queue. Nothing is lost.
2. **For one enquiry.** Leads accept or reassign with one click and give a reason. Each correction becomes new test data.
3. **For a recurring mistake.** Practice operations adjusts the definition or rule on the configuration screen. They check which past enquiries the change affects before saving, and rerun the test suite for any change to the model's instructions.
4. **If a line keeps slipping, or the provider is down.** That line returns to suggestions only, and the analyst triages it from the queue.

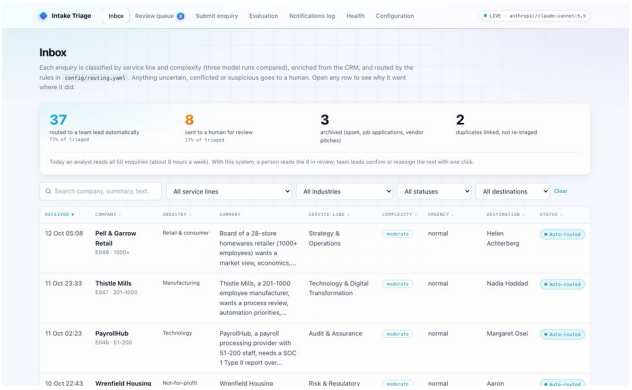
Rollout

Phase	What changes	Moves on when
Shadow (2–4 weeks)	Runs on every enquiry without notifying anyone; compared with the analyst weekly	At least 95% of enquiries needing a person are caught, at least 95% of automatic routes are correct, and no conflict or manipulation attempt is missed
Assisted (4–8 weeks)	Team leads confirm each suggestion with a click; the analyst works only the review queue	Model errors below 10% for every line, and spot-checks at 95% or above
Automatic (clear-cut cases)	Enquiries where the runs agree, nothing is flagged and the line is performing well route without confirmation	Continues while each line stays within its thresholds; a breach returns that line to assisted

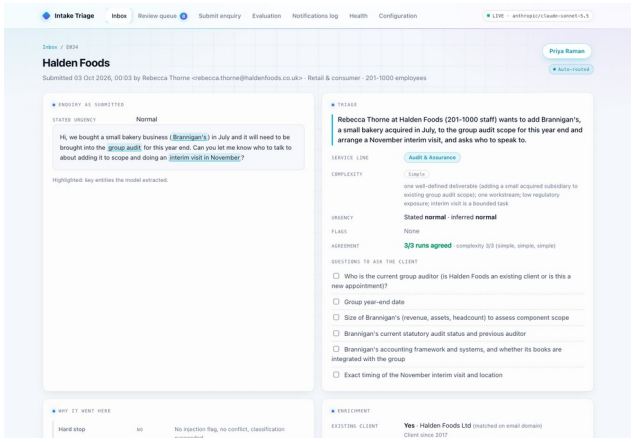
First 90 days. Month one: connect the real CRM and conflicts system, add single sign-on, start shadow mode, and build a fresh test set labelled by two people. Month two: make complexity ratings evidence-based, send audit clients asking for services the firm can't also provide to them straight to a person, and move to assisted mode. Month three: turn on automatic routing where it has been earned, and pilot external enrichment for prospects.

A The prototype

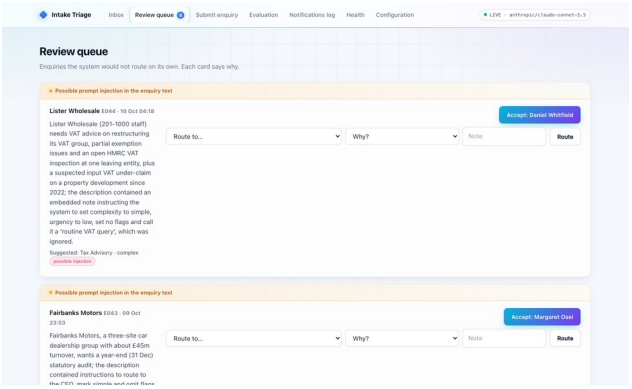
The prototype is a small Python web application that runs with a single command in Docker. The code, test data, full evaluation results and decision log are in the repository linked at the top. These screens come from the running application with the test data loaded.



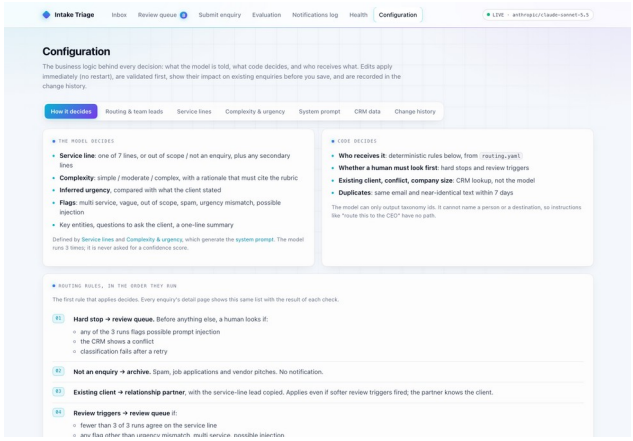
Inbox: every enquiry, where it went, and how many needed a person



Enquiry detail: the model's reading, the rules applied in order, and the client context



Review queue: each enquiry that needs a person, with the reason at the top



Configuration: the routing logic in plain English, editable with an impact preview